ORIGINAL CONFERENCE

Task-Authoring Workflow

Every step of building a HarmonyGames task that passes — the injection, the prompt, the hardness check, the Oracle Events and the rubric, in the order you do them.

Distilled from the 13 project guidelines · Universe date: October 1, 2025 or February 14, 2026 (00:00 UTC)

0 · The one rule, and the loop

A task is a realistic work request from one of HarmonyGames' 19 staff personas, run **six times by Opus 5.5** and graded by the rubric you write. You win when the work is **hard to find but has exactly one correct answer once found**.

THE CORE RULE

Hard is fine. Unfair is the problem. Difficulty must come from the **environment** — information scattered across services, some of it stale — never from a vague prompt. A task nobody can read one way fails Unique Ground Truth. A riddle that hints at the trap is contrived, not hard.

The nine parts, in review order

Reviews run in this sequence; **PASS and NON-FAIL both let you continue, a FAIL blocks.** Change something upstream and you must re-run every review that reads it.

0	1	2	3	4	5	6	7	8
Explore	**Design**	**Inject**	**Prompt**	**Hardness**	**Oracle Events**	**Rubric**	**Verify**	**Submit**
`universe`	`decision map`	`/injection`	`/prompt`	`/hardness`	`/oe`	`/rubric`	`6 runs`	`pass screens`

What makes a task hard (build these)

DO

- Answer needs facts combined from **two or more services**; no single source is enough.
- The override is **not** “newest message wins” — the correction hides in a thread reply or a doc edit.
- Hidden evidence changes a **status, number or write** — not just a comment.
- The final answer is **calculated** from records; no record states it.
- Rules live in the **universe** (a manager's message, a policy doc), not the prompt.

AVOID

- Routine traps Opus already checks (stale-vs-newer, obvious decoys).
- Hard evidence that only changes a comment — if missing it, the result is unchanged.
- The answer sitting in **one message**, often the channel the prompt names.
- Rules or numbered steps spelled out in the prompt.
- Adding **more rows of the same** mechanical work. Never add ambiguity to fake difficulty.

ASK FOR HELP EARLY

When a review fails, ask the portal assistant: “Help me fix the failed items in my latest /prompt report.” It works from what you meant. Use `/admin` only for platform problems (a crashed run, a broken pass card). Keep moving — go quiet too long and your claim can lapse to someone else.

0

universe explorer

Explore — find a real decision

Check your assigned persona and universe date first. There are two dates: **Oct 1, 2025** and **Feb 14, 2026**. In the Feb 14 universe the studio announced its shutdown on Feb 11, so business-as-usual premises are wrong there. A task built on the wrong date fails.

Let the portal assistant search every service for you. Look for:

- a real decision your persona would make, with at least one **write** (ticket, message, doc or sheet);
- evidence spread across **two or more** services;
- places where the base data **already disagrees** — stay away from grading those facts.

1

decision map

Design — plan before you write

Write a short decision map before any SQL or prompt:

1 · The question	What decision or result does the task need?
2 · Deciding facts	The 2–4 facts the agent must know to get it right.
3 · Where each lives	Service, record, date.
4 · Careless miss	Which fact a careless agent misses, and what wrong result that produces.
5 · One answer	What in the data settles each fork, so a careful agent reaches exactly one result.

> **TERMINOLOGY**
> Call it **data seeding** or **injection**. “SQL injection” means a security vulnerability and is not what this is.

2 /injection

Seed the data

Add the records your task needs **without breaking the universe around them**. Seed the **symptom and the evidence, never the conclusion**. Your injection SQL is the **only record** of what you changed; a reviewer re-injects it into a fresh environment, so if it doesn't reproduce your universe exactly, the task can't be re-run and is returned. (You also paste the SQL into the task yourself — passing `/injection` only validates it.)

The six consistency rules

Rule	What it means	Failure signature
On the clock	Every date before your universe date; `created` never post-dates `updated`.	Temporal field inconsistency
Consistent across services	Anything referenced somewhere exists where it is referenced.	A Slack thread about a PR not in GitHub
Coherent	Figures reconcile with the records they derive from.	A doc says $65 against a sheet saying $35
Structurally valid	No invented fields; IDs follow the existing format.	An ID breaking `ENG-####` / `ZOM-###`
Reachable & findable	Your persona can read it; a realistic search path reaches it.	Unsolvable task, or evidence nobody can find
Natural, not pre-solving	Seed the symptom and evidence, never the conclusion.	Pre-solved prompt, or AI-flavoured text

TWO SILENT DEFECTS, AND TWO HARD CHECKS

Watch **state** (a ticket Closed whose PR never merged; a task Done with an open blocker) and **semantic** drift (one programme under two names — fine only if you intend the agent to reconcile it, and you say so in Oracle Events). Before running: every figure/date/status matches base records, every reference exists, people are employed and in-channel on that date, new IDs come after the base maximum, and **no notes to the solver inside records**.

GATE · `/injection` returns PASS or FAIL. A FAIL blocks — fix the SQL and rerun. Snowflake can't be injected: it is a separate analytics warehouse, read-only reference.

BEFORE YOU LEAVE INJECTION

- ■ Every figure, term, date and status matches the base records on that entity.
- ■ Every record mentions only things that existed at its timestamp.
- ■ People are employed and in the channel on that date; one existing account each.
- ■ Every copy of a changed fact is updated; every referenced ID exists; new numbers come after the base maximum.
- ■ No notes to the solver inside records. Forks are settled with a seeded correction or a clear precedence.
- ■ `/injection` returned PASS.

3 `/prompt` Write the prompt

One plain request from the persona that a careful agent can read **only one way**. The shape that works: **who asks** → **what's wrong (a visible symptom)** → **investigate** → **act** across two or more services, with the destination and any deadline named.

LLM USE IS PROHIBITED

Prompts must be written by **you, from scratch** — not drafted or polished by any LLM. An AI-text detector runs on your prompt before anything else. Write how you actually type to an assistant: clean spelling is fine; staged typos and emoji read as contrived.

ALWAYS

- Require real tool use; span 2–3 services.
- Sound like a real person typing: first person, informal.
- State dates absolutely, anchored to your universe date.
- Ask for **investigation and action**.
- Name where the output goes, when the person would know it.
- Keep every sentence load-bearing.

NEVER

- Name a tool, a parameter or an MCP server.
- State the root cause (that pre-solves it).
- Include internal IDs a person wouldn't know.
- Announce an injected failure or an async wait.
- Format it like a document, spec or step list.
- Pose a riddle, hint at the trap, or add an answer-key clause.

The prompt can only mean one outcome (the #1 failure)

Write the outcome you want and the best **other** outcome a careful person could defend from the same prompt. If they differ on **what gets decided, what gets written, where it goes, or what it must say**, find the sentence that rules the other out — or add one. Settle these five in the prompt: which items are in/out, which number wins, which source wins, what counts as the event, and which single action happens.

GATE · `/prompt` returns PASS or NON-FAIL (treat NON-FAIL like PASS). A FAIL blocks. With hinting removed, nothing downstream rescues an unsolvable prompt.

4 /hardness
Make it hard enough

Prove Opus 5.5 can get it wrong **for the right reason**, then show it with the run. Your difficulty mode sets the ceiling. **Start the six-run baseline the moment `/prompt` passes** — waiting is where claims lapse — and write Oracle Events and the rubric while it runs. Don't tune the prompt to the baseline: revising until it passes destroys the only difficulty signal the task has.

The difficulty ceiling

Difficulty mode	Average weighted rubric score must be ≤	pass@1
Easy	90%	≤ 67% (no more than 4 of 6 runs pass everything)
Medium	85%	≤ 67%
Hard	70%	≤ 67%

Four hardness tests (ask the assistant each)

Test	Ask	What it means
Shallow path	“What answer do I get from only the newest message in the channel?”	If that's the correct answer, the task is too easy.
Single record	“Does any one record state the final answer or verdict?”	If yes, split the fact across records or reword.
Decision flip	“If an agent missed the hard record, what changes in the output?”	If only a comment changes, move the evidence onto the decision path.
Fork	“Can a careful reader reach two different answers?”	If yes, settle it in the data.

HOW THE TOOL-CALL MINIMUM IS COUNTED

If any of your six rollouts scored **100% on your rubric**, that rollout is a **golden run** and the tool-call minimum applies to it alone. If none did, it applies to the **average across all six**. A golden run is model-produced — your Oracle Events do **not** enter this calculation. Padding doesn't help: blocked and repeated calls add nothing, and an inflated count is itself a failing error.

BEFORE YOU LEAVE HARDNESS

- ■ Shallow path doesn't give the answer; no single record states the verdict.
- ■ Missing the hard record changes a status, number or write — not just a comment.
- ■ A careful reader can't reach two answers.
- ■ You know your mode's ceiling (Easy 90% / Medium 85% / Hard 70%); pass@1 ≤ 67%.
- ■ The Opus 5.5 baseline ran on the exact current prompt and SQL, all six completed.

5 /oe Oracle Events

The steps a correct agent takes, from the first thing it finds to the last thing it writes. They **prove the task is solvable** (hinting is gone, so this is the safety net) and they **drive the rubric**. Write them by hand — the field blocks pasting. Plain text, numbered, a few lines each.

Five fields per event

Field	What goes in it
Action	What needs to happen — search, retrieve, send, create, update.
Information	What must be discovered or verified before the agent can act.
Tool / Service	Which tool and service, matching where the data actually lives.
Parameters	The important inputs: identifiers, filters, date ranges, values.
Expected data	What the step should retrieve or produce — the real values.

TWO THINGS ORACLE EVENTS NEVER DO

No non-actions — “the agent does not stop early” is not an event; write “reads the thread to its end. Expected: …” instead. **No final-response content** — what the agent ultimately reports is the rubric's business; convert “the response explains…” into the discovery step that found it. Every forced error or async wait gets its own explicit recovery/wait event.

GATE · `/oe` passes or blocks — no NON-FAIL. Evals are deterministic, so re-running unchanged returns the same verdict. Fix anything it flags.

BEFORE YOU LEAVE ORACLE EVENTS

- ■ You wrote every event yourself; nothing was generated or pasted.
- ■ Every event is an observable action or discovery; none describes the final response.
- ■ Every event names its service and its expected data; every parameter/ID/figure checked against the explorer.
- ■ Reading the set straight through produces a trajectory that would actually work.
- ■ Every required write the prompt asks for appears; forced errors / async waits appear as explicit events.
- ■ `/oe` returned PASS.

6 /rubric
Write the rubric

Grade **exactly what the prompt asked for**, one check per row, derived from your Oracle Events. **Write every criterion yourself** — prefill is off and the builder opens empty on purpose; `/rubric` only **audits** what you wrote. Phrasing: **Agent + active verb + the exact value**. Tool names must never appear.

Two categories, weights by difficulty

Outcome (the default): verifiable from the end state. **Process** (the exception, almost always ordering): only when all three conditions hold — required by every valid path, no stricter Outcome can capture it, and it's a behavioural property not a trace. Zero Outcome rows fails; >50% Process fails; two *unnecessary* Process rows fail.

Weight	Difficulty axes exercised	Example
1	None or one	A value stated directly in one place; confirm a named file was created.
3	Exactly two	Compare two sources and surface a mismatch; derive one field.
5	Three or more	Reconcile a figure across three sources and apply a rule from a policy doc.

WEIGHTS MEASURE DIFFICULTY, NOT IMPORTANCE

“Agent emails the right person” may be the whole point of the task and is still weight 1 — the address is in Contacts. The high weights belong on reconciliations and derived figures.

EVERY ROW

- Starts with “Agent” + an active verb; affirmative even when the prompt forbids an action.
- Self-contained: names the answer, not just the question.
- One verifiable claim; one fact per row.
- Carries criterion, justification, evidence — evidence cites the record and matches the value.

NEVER

- A disjunctive “a doc or a Linear list” (phrase broadly, or accept both).
- Grade unstated expectations — format, line count, a time zone the prompt didn't require.
- Hard-code a conclusion from ambiguous evidence, or grade the distractor.
- Two criteria on one fact (Moderate); the same fact in 3+ rows (fails).

7

verifier · 6 runs

Verify — read the matrix

Run the rubric verifiers once; each of the six runs is graded against your rubric, one column per criterion. **A column red in every run is an all-fail** — justify it or fix it. Rate a rubric all-fail only when it failed **every completed run**. Before judging difficulty, fix every unfair row and re-grade.

The difficulty check

Check	Target
Difficulty bar	Average weighted score within your mode's ceiling (Easy 90 / Medium 85 / Hard 70).
pass@1	≤ 67% — no more than 4 of 6 runs pass everything.
Failure patterns	At least 2; no single pattern covers more than half.
Tool calls	Above your minimum — on a 100% run if one exists, else the six-run average.
Completed runs	All six.

NEVER DELETE ROWS TO MOVE THE NUMBERS

Deleting rubric rows to pass the difficulty check creates a missing-coverage defect and leaves the task exactly as easy. Fix the cause in the data, re-run upstream, and re-grade.

8

pass screens

Submit

Submit the moment the last review passes — some CBs passed every review and still lost the task by not submitting in time. Enter each verdict, its score (1 for PASS/NON-FAIL, 0 for FAIL) and the verification code from the pass screen. Screenshot the whole pass screen with the code visible.

THE SUBMIT CHECKLIST

- Scenario fits every assignment axis, with your assigned persona and universe date.
- SQL ran in Database, sits in the Injection SQL field, and `/injection` reached at least NON-FAIL.
- Prompt is your own words: a plain request, no tool names, nothing pre-solved, one outcome.
- Every write has an Oracle Event and an Outcome criterion; no fact repeated across rows.
- `/prompt`, `/hardness`, `/oe` and `/rubric` reached at least NON-FAIL, with each verdict and verification code entered.
- The difficulty check passes for your mode, and every all-fail is valid.

Reference — the rules you check against

Persona read scope (ACL)

The assigned persona is **fixed** (switching fails). A denied read returns **“not found”**. **Writes are governed by the tool catalog, not read scope.** Universe Explorer is author god-mode — never build on, or grade against, evidence only you can see.

Class	Services	Behaviour
Unscoped — shared	GitHub, Trello, Linear, Contacts	Every persona sees the same records.
Scoped — filtered	Gmail, Slack, Google Calendar, Drive, Docs, Sheets, Slides	`list` / `search` / by-ID all filter to the persona.
Unclassified — verify	Confluence, Snowflake	Open from the persona's view and confirm it returns before you rely on it.

How a task is scored

Five dimensions, each 1–5. **The lowest dimension is the task score** — a strong score elsewhere can't make up for a failing one. 1–2 = FAIL, 3–4 = PASS, 5 = perfect.

Dimension	1–2 (FAIL)	5 (perfect)
Unique Ground Truth	Rubric accepts only one of two fair readings.	Every expert agrees.
Agent-centric phrasing	Passive, or names a tool.	Clean throughout.
Negative criteria	A criterion says what the agent must *not* do.	Every criterion says what the agent does.
Prompt clarity	Vague, or two write paths.	Unambiguous.
Truthfulness	One major or two minor errors.	All verified.

PROMOTION & LIFECYCLE

Two consecutive tasks scoring 4+ promote through to delivery. A score of 2 is fixable (visible potential, not a removal). Two consecutive scores of 2 or below — or suspicious behaviour or spam — disable the account.

Rubric defect thresholds

Count each row once at its highest severity. Each line below fails the task on its own:

The rubric fails with more than	Of the rubric's rows
Major issues	10%
Moderate or worse	15%
Minor or worse	20%

What to re-run after a change

A pass only covers the inputs that review read. Change something and re-run every review that reads it, in order. The Opus run must use the exact current prompt and SQL — an old run doesn't count.

You changed…	Re-run
Injection SQL	/injection → /prompt → new Opus run + /hardness → /oe → /rubric
Prompt	/prompt → new Opus run + /hardness → /oe → /rubric
Task details (assignment)	/prompt → /oe → /rubric
Oracle Events	/oe → /rubric
Rubric	/rubric
Universe date	everything, from /injection

THE AUDIT, IN ONE LINE

Rubrics reject correct answers **more often than they accept wrong ones** (44% vs 29%), and four out of five defects originate in the rubric itself. The most common prompt failure is two defensible outcomes; the most common rubric failure is missing coverage. Write the Oracle Events first — a complete set is the cheapest protection against both.

Appendix A — The universe

HarmonyGames is a founder-led mobile game studio — small enough that one person's Slack DMs matter, large enough that nobody has the whole picture. The information an agent needs genuinely exists and is genuinely scattered. **19 staff personas** across product, engineering, design, data, marketing, finance and operations; **13 connected services**.

Fact	Detail
Flagship title	A domino-matcher (Domino Delights) — the established revenue source.
Newer title	A zombie match-3 (Zombie Match 3D) — still finding its audience.
Third title	A fighting game (Combo Fighter) — most recently launched; prototype-scale data.
Universe date	Oct 1, 2025 (a Wednesday) **or** Feb 14, 2026 (a Saturday), 00:00 UTC. In the Feb universe the shutdown was announced Feb 11, 2026.

The thirteen services

Service	What lives there	Good for
Slack	Channels, DMs, running commentary	Richest source of evidence and distractors — most tasks start here.
Gmail	External correspondence, publishers, investors	The outside view, which often disagrees with the inside one.
Google Drive	Shared files and folder structure	Where a written deliverable is expected to land.
Google Docs	Design docs, specs, legal, post-mortems	Long-form intent the data can contradict.
Google Sheets	Tracking sheets, budgets, financials	Numbers that must reconcile against Snowflake and each other.
Google Slides	Pitch decks, quarterly reviews	Summarised figures that go stale the moment data moves.
Google Calendar	Meetings, availability, scheduling	Ordering evidence; a natural home for a deadline.
Contacts	People lookup, email addresses, roles	Resolving “tell whoever owns it” into a real recipient.
GitHub	Code, PRs, reviews, release history	Ground truth about what actually shipped and when.
Linear	Engineering tickets, bugs, projects, cycles	Both a source and a destination — most tasks end in a ticket.
Trello	Roadmap, feature and prototype boards	Product-side planning, often out of date relative to Linear.
Confluence	Team wiki and knowledge base	Policies and conventions an agent should apply. **Verify from persona view.**
Snowflake	Data warehouse, analytics tables	The only place many figures are **derived** rather than read. **Read-only.**

THE CLOCK

The universe is frozen at your universe date; everything before it is the past, nothing after has happened. **Write dates absolutely** (“February 20, 2026”, never “next week”). Nothing may post-date the clock; `created` never post-dates `updated`. Business messages at 3am or a weekday standup on a Sunday read as synthetic. A deadline *may* fall after the clock. People read Pacific; SQL timestamps are UTC (+00) — when the day differs, write both (“Nov 16 PT / Nov 17 UTC”).

Appendix B — Tool behaviour & the six difficulty levers

The depth bar — four things a real task needs

Requirement	What fails it
Real tool use	A question general knowledge can answer without touching the universe.
Genuinely cross-service	One service resolves the whole thing, or a second is touched only trivially. (2 is the floor; 3+ is the target.)
Investigation and action	A pure lookup, or a pure write with nothing to work out first.
Not pre-solved	The prompt states the root cause, leaving the agent nothing to diagnose.

The six difficulty characteristics — all optional, all stackable

Characteristic	Definition	Built by
Ambiguity	Prompt under-specifies; the agent resolves gaps from the environment.	Seeding
Dynamic change	What the agent learns early is overturned by what it finds later (order matters).	Seeding
Multi-source	Info split across 3+ sources, with distractors and a disagreement.	Seeding
Async tool calls	Submit-then-wait tools; the agent parks the dependency and resumes.	Config
Error recovery	Forced tool failures; the agent retries, works around or escalates.	Config
Access control	The agent must work within what its persona may read.	Persona + seeding

Tool behaviour worth knowing before you build

Behaviour	Consequence for your design
Search is fuzzy; fetch is exact	Naming an exact document/thread/ID in the prompt collapses discovery to one fetch — usually the opposite of what you want.
Async waits affect search only	A matching fetch stays instant; an async wait (submit → check → get, 30–35s/call) only bites when discovery is genuinely required.
A forced error looks like an ACL denial	On a scoped service the agent can't tell them apart. Don't force an error on a tool whose denial the agent is meant to reason about; prefer unscoped (GitHub, Linear, Trello, Contacts).
Snowflake answers are derived	Figures usually need a query plus an aggregation — which is why Snowflake criteria tend to carry high weights.
Blocked & repeated calls still count	They add nothing to real depth, and an inflated tool-call count is a failing error in its own right.

FORCED ERRORS — READ THE ERROR RATE

Error rate 1.0 (default): the tool fails on *every* call — a permanent outage, a retry will *never* succeed (not marked transient). Tests a workaround or escalation. **Error rate below 1.0:** fails that proportion of calls, marked transient — a retry *can* succeed. Tests retries. Either way the task must still be solvable: name the alternative path, confirm it exists, and put it in your Oracle Events. Reference files are optional — upload a folder only if the agent must read an attachment, and upload the same files again on the next step (that copy is the record).

Appendix C — Injection schema & the reconciliation traps

The Injection Schema guideline is the per-table column reference. You write **INSERT/UPDATE** SQL against the service tables (Gmail, Slack, Linear, GitHub, Google services, Trello, Contacts, Confluence). **Snowflake (`ANALYTICS`) is read-only reference — you never write SQL for it;** it is listed so you don't have to. Match every column name, type and ID format exactly; invented columns fail.

Snowflake ANALYTICS — the schemas

Schema	Holds	Join key
RAW_TELEMETRY	Event firehose (RAW_EVENTS) + 8 derived projections; 6 are session-sampled.	player_id / user_pseudo_id → PLAYER_DAILY_PROGRESS.player_id
MARKETING	Installs, UA spend, app-store reviews, attribution.	install_id; customer_user_id = player key
MONETIZATION	IAP transactions, daily revenue, ad revenue, virtual-currency ledger.	transaction_id; player_id
FINANCE.EXPENSES	Monthly burn, cash balance, headcount, payroll roster, finance ledger.	employee_id (not unique); roster_line_id
LIVE_OPS.EVENTS	Push-notification stats, live-event performance (827 runs).	event_id; push_id

SAMPLING — WHY RECOUNTS WON'T MATCH

RAW_EVENTS is the full stream; derived tables are deterministic session samples (RAW_GAME_EVENTS / STG_EVENTS_ENRICHED 1:8; PROGRESSION / SESSION_LEVEL / AD_IMPRESSION_EVENTS 1:16; DESIGN_EVENTS 1:64). RESOURCE_FLOW_EVENTS, CLIENT_ERROR_EVENTS are **not** sampled. **Aggregate marts were computed before sampling**, so a recount from a sampled table will not match the mart total — don't build a graded fact that assumes it does.

Built-in reconciliation traps (grade the distinction, not a recount)

Trap	The distinction
IAP revenue vs receipts	IAP_TRANSACTIONS is NET of refunds (977 rows, a strict subset); STORE_TRANSACTIONS is GROSS (993 = 977 + 16 refunded). Share transaction_id — use STORE for receipt counts, IAP for revenue.
Ad revenue vs impressions	AD_REVENUE.impressions = BILLABLE (what the network paid on); AD_IMPRESSION_EVENTS is the client-side 1:16 stream. They reconcile on MONEY, not counts — don't truncate either side to make them agree.
Runway	Already a known base-data conflict: investor letter says ~4 months, CASH_BALANCE says 3.1 (Aug 31) / 2.7 (Sep 30). Use CASH_BALANCE or leave runway out — don't grade a disputed figure.
Headcount	MONTHLY_BURN/CASH_BALANCE count non-founder PAYEES (peak 16); non-founder people peak 19 (full-time + contractors); the three founders are excluded. “21 including contractors” is a reported band top, not a realised count.
Combo Fighter scale	Prototype-scale by design (~3% of events; absent from some economy tables). Absence of gem rows is a modelling boundary, not evidence gems were unused.

Appendix D — The prompt-failure catalog

From 340 prompt reviews (104 failed). A score of 1–2 on any requirement fails the task; the task score is the lowest it got. **The test before you submit:** write the outcome you want and the best other outcome a careful person could defend. If those differ on what's decided, written, where it goes or what it must say, find the sentence that makes the other impossible — or the task fails.

In plain words	Failed	What the fail looks like
One right outcome	66 / 104	Two different outcomes both fit the prompt.
Clear what gets written	46 / 104	The two outcomes change what gets written.
Needs two services	~40 / 104	The answer sits in one service.
Stated facts are true	11 / 104	A channel, amount or date in the prompt is wrong.
This person can do it	9 / 104	The persona can't plausibly do the main request.
Sounds like a colleague	8 / 104	It reads like a spec or a step list.
The fact exists	4 / 104	A needed fact is not in the records.
This person can open it	3 / 104	A required record is hidden from this persona.
The cause is left to find	1 / 104	The prompt already states the cause.

The five checks a reviewer uses to keep a second reading alive

Check	What it asks
Newer source	Is there a later record of the same fact? A later update can replace an earlier one; a missing later record does not.
Authority	Did the prompt say to follow a particular source? Quote that sentence.
Who said it	Is it the person or file the prompt named? A different person's words aren't automatically the same fact.
Scope	Does this record settle the whole task, or only one part?
Same result	Do both readings end with the same decision, writes and required contents? Same destination isn't enough if contents differ.

> **WHEN MORE THAN ONE ANSWER IS ACCEPTABLE**
> Say so in the prompt and state the boundary in plain words. Then the Oracle Events and rubric must accept **every** answer inside that boundary — they cannot quietly require one. A later rubric row that requires one of two live readings is wrong: fix the prompt, or write the boundary and grade both.

Appendix E — The rubric-failure catalog

From a 37-task blind audit, 298 rubric issues (only 3 tasks passed all sections). Count each row once at its highest severity; a missing outcome counts against the rubric but is not added to the row count. **Thresholds:** more than **10%** major, more than **15%** moderate-or-worse, or more than **20%** minor-or-worse — each fails the task on its own.

Defect	Share	What it looks like
Incorrect	22%	The criterion asserts something the universe contradicts — runs read the signed doc, reported right, scored zero.
Non-atomic	22%	One row bundling several independently-variable assertions.
Missing coverage	21%	A required result has no row; an incomplete answer scores full marks.
Unauthorized	6%	The rubric invents a requirement — often detectable because the task's own solution fails the row.
Overly specific	5%	Right substance, rejected over wording (“LET GO” instead of “HAND OFF”).

TWO FINDINGS TO INTERNALISE

Rubrics reject correct answers more often than they accept wrong ones — 44% of issues punish a good agent vs 29% that let a bad one through. And **four out of five defects originate in the rubric itself**, not downstream of a bad solution path. Excellent Oracle Events do not save a sloppy rubric.

All-fail rubrics, and what fails the set on its own

After the verifier runs, rate any rubric that failed **every completed run** an all-fail (if 4 of 6 completed, it's all-fail only if the model failed it on all four). Before justifying, ask: is it self-contained, atomic, agent-centric, grounded; flexible enough for valid alternatives; actually required; and — if you forced an error or set an async wait — does the recovery/slow path really exist? If any answer is no, **fix the rubric rather than justify it.**

Situation	Severity
Two or more invalid all-fail rubrics	Fail
One invalid all-fail rubric	Non-fail
A valid all-fail rubric with an unfactual/irrelevant justification	Non-fail

Fails the set on its own: zero Outcome rows; more than half Process, or two unnecessary Process rows; the same fact graded in 3+ rows; a title that isn't about the agent or names a tool; a row that says what the agent must *not* do; a row missing its title, category, weight, justification or evidence; two rows that every careful attempt fails because the value isn't in the records.

Appendix F — The worked example (Combo Fighters launch)

The ML team's chosen high-quality example — a Long-Horizon task, all real output. **Persona:** Marcus Bennett, Artist (Design). **Competency:** Marketing & GTM (he made the marketing videos and the playable VFX, so GTM went through him — a defensible, non-obvious match). **Skill:** Analysis & Synthesis, stacked with Retrieval and Actions. **Characteristics:** Ambiguity, Dynamic Change, Multi-Source. **Services:** Slack, Snowflake, GitHub, Confluence, Linear, Trello. ~10h human estimate.

The prompt (why it works)

> *"Robert wants the Combo Fighters launch written down properly before Leonard archives everyone's Google accounts on February 15, and honestly I don't think anybody actually knows what state it's in… can you work out what really happened and what is still genuinely open? Then put the record somewhere proper next to the rest of the launch stuff and raise tickets for whatever someone needs to pick up."*

It reads like real typing; names nothing (a deadline and a feeling); has two phases (investigate, then write + raise tickets); leaves the trap hidden; justifies the persona; is not pre-solved; and has unique ground truth (route varies, destination doesn't). **Feb 15 is absolute and one day after the clock** — a deadline, not something already done.

What was seeded — dynamic change done right

One Slack message from Robert (the CPO), dated Feb 13, is the **newest, most authoritative** summary — and both halves are wrong: "we ran it across all the networks" (meta was the only network that bought an install; the other six names are monetization placements) and "the playable is the only thing I'd call unfinished" (the playable is the one creative that shipped). The agent must overturn the most confident summary in the universe. Robert being wrong is realistic — and truthfulness applies to what *you* assert, not what a character believes.

Three of the nine Oracle Events

```
OE 2  Establish which build the January approval message refers to.
      slack_conversations_history (channel C09UHHN6PFZ, 2026-01-17..20)
      Expected: 1.0.56 posted 01-17; approval 01-19 06:41; 1.0.57 submitted 06:44, released 01-20.

OE 4  Aggregate paid spend by network for the title.
      Snowflake AD_SPEND_DAILY, filter game_id = combo_fighter
      Expected: 152 rows, channel = meta on all, 4510.33 USD, 615 paid installs, 2025-12-26 to
2026-02-08.

OE 7  Read the 2026-02-11 playable thread to its end, not to 21:08.
      Expected: rejection 21:03, workaround 21:10, "it works" 21:22, campaign 21:37, iOS-only
confirmed 21:54.
```

Each names the service, the filter and the expected values. **OE 7 is the whole task in one event** — "to its end, not to 21:08" captures exactly where a careless agent stops. No event describes the final response; no non-actions.

The rubric and the baseline

31 criteria, all Outcome (no Process needed — every requirement was provable from the final record). Weights ran 1 (a stated build/date), 3 (one aggregation or one thread read past its apparent conclusion) and 5 (reconcile monetization-vs-acquisition across Slack/GitHub/Snowflake; establish 1.0.57 not 1.0.56 from three timestamps with a three-minute gap). The criterion that carries the task's whole point — a record exists in the right place — is weight 1; the reconciliations are the 5s.

Baseline	Result
Model / rollouts	Opus 5.5 / 6, all completed. Best rollout 18 of 31.
pass@1	0 — no run passed everything.
Totals reported	Every one correct — yet both consequential findings inverted (playable called unfinished, title called live).
Where difficulty sat	Not volume (12,100 messages handled fine) — **knowing when not to stop**. The thread reads done at 21:08; it continues to 21:54.

THE SAME TASK, DONE BADLY (the contrasts)

Prompt: formal LLM-sounding wording; naming `slack_conversations_history` + a raw channel ID (tool mention, Fail); “Robert thinks we ran ads on every network but…” (pre-solved, Fail); “create a Google Doc… or just summarise it back” (two valid homes, Fail); a bolt-on meeting request with “next week” (relative date, Fail). Oracle Events: an OE with no Expected line; naming Sheets for a figure that's in Snowflake (inaccurate); “the agent does not stop at the first message” (non-action).

Appendix G — Glossary

Term	Meaning
Fork	A place where a careful agent could reach two different correct results.
Unique Ground Truth	The check that only one result is correct. Forks fail it. Scores 5 or 1–2, no middle.
Borderline item	A record sitting between two labels or two rules. Check these first.
Decoy	A record that looks like the answer but isn't (older, withdrawn, other team).
Seeded record	A record you add with your injection SQL.
Golden run	A model-produced rollout that scored 100% on your rubric; sets the tool-call minimum. Not your Oracle Events.
Oracle Events (OE)	The hand-written steps a correct agent takes; prove solvability and drive the rubric. Replaced the old “golden solution.”
Outcome / Process criterion	Outcome = verifiable from the end state (default). Process = ordering a final state can't prove (the exception).
Persona visibility	Whether your persona can see a channel, mailbox or file on that date.
Advisory / Minor	Noted by the review but doesn't fail the step. No fix needed.
NON-FAIL	Passes the step — treat it like PASS.
pass@1	Share of rollouts that pass every criterion. Must be ≤ 67%.

ONE LINE TO CARRY

Seed the symptom · write it yourself · one outcome · Outcome first. Every task is different — verify every name, ID, date and figure in your own universe.